

Raster Images



What is an Image?

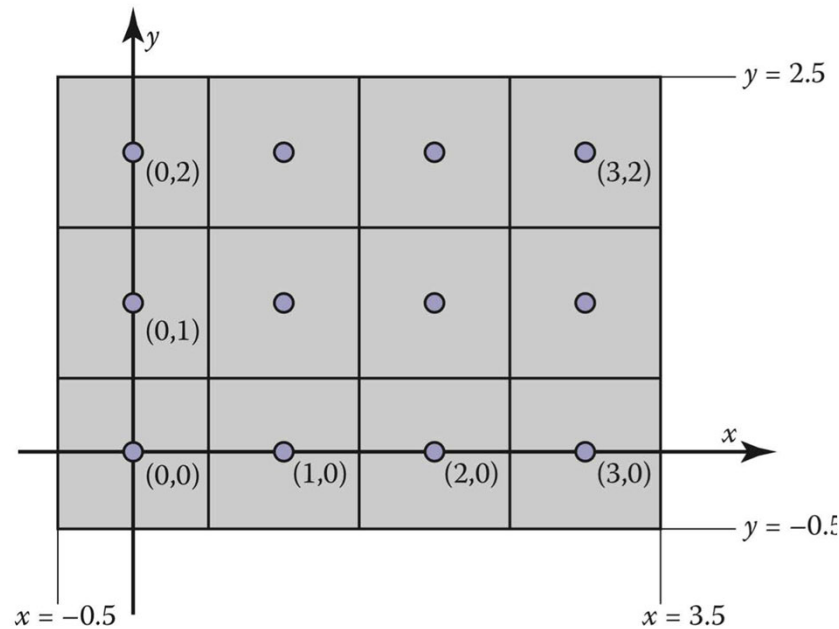
Image = distribution of light energy on 2D “film”

Digital images represented as rectangular arrays of **pixels**



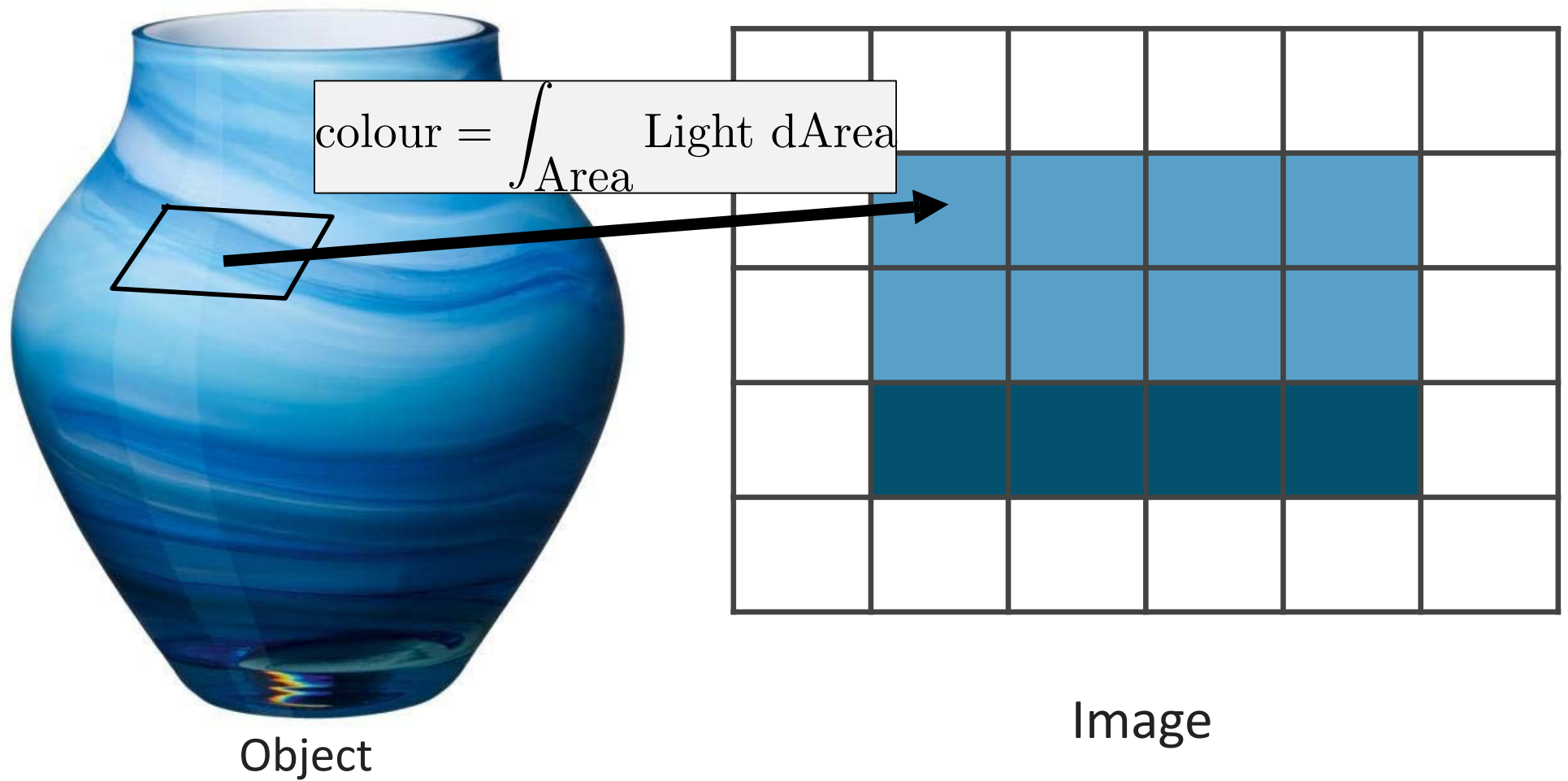
Image = a function defined for an array of pixels

$$I(x, y) : \mathbb{R}^2 \rightarrow \mathbb{R}^{+n}$$



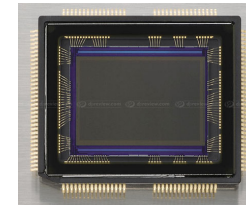
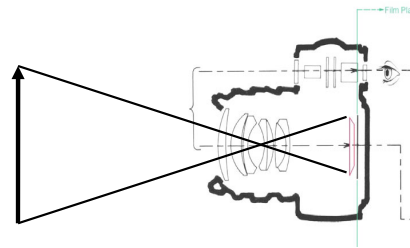
Pixel Co-ordinates

A Pixel in 3D need not be 'square'

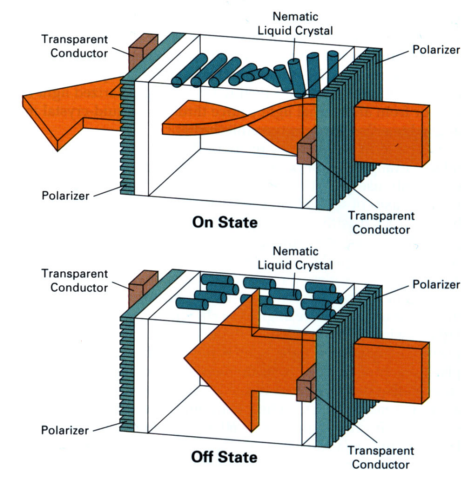


Raster Devices and pixels

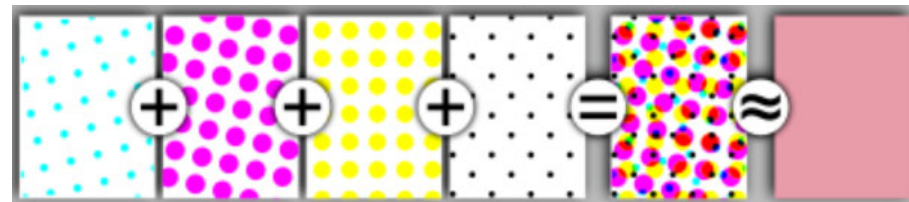
- Input (scanners, cameras)
2D array sensor: digital camera



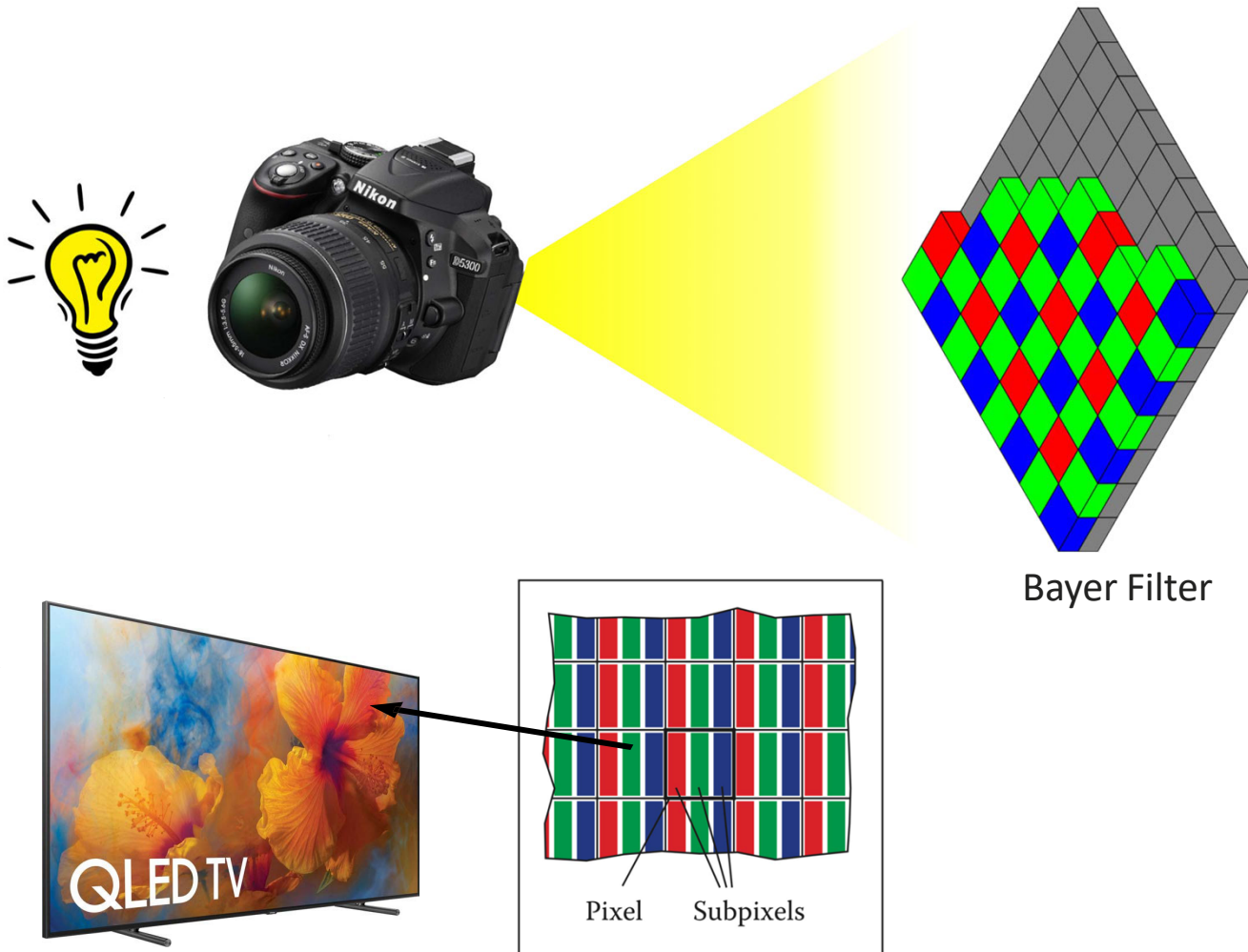
- Output (printers, displays)
Emissive: light-emitting diode (LED)
Transmissive: liquid crystal display (LCD)



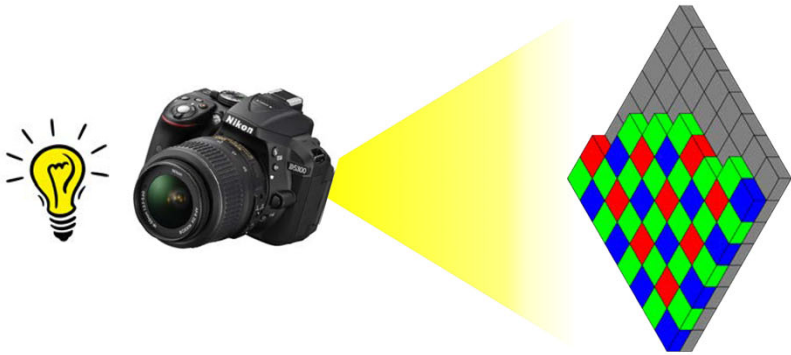
Ink-jet printer



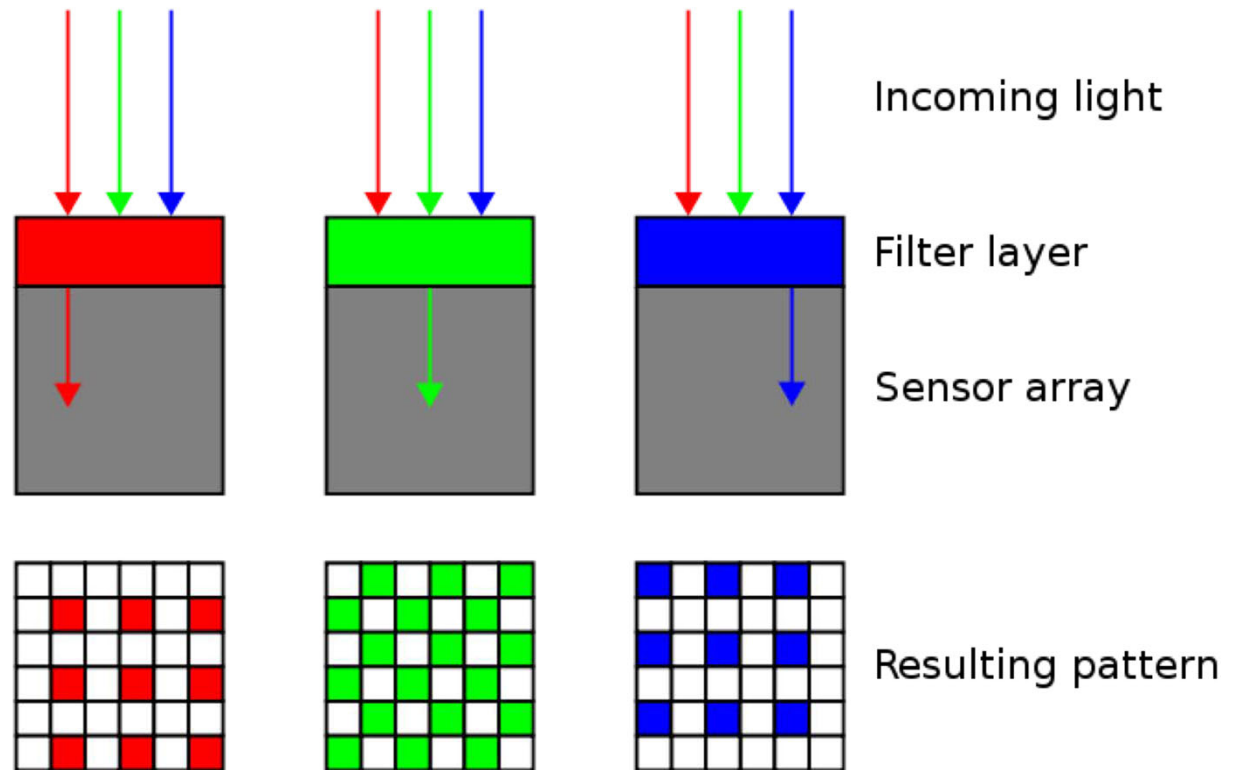
Raster Input/Output



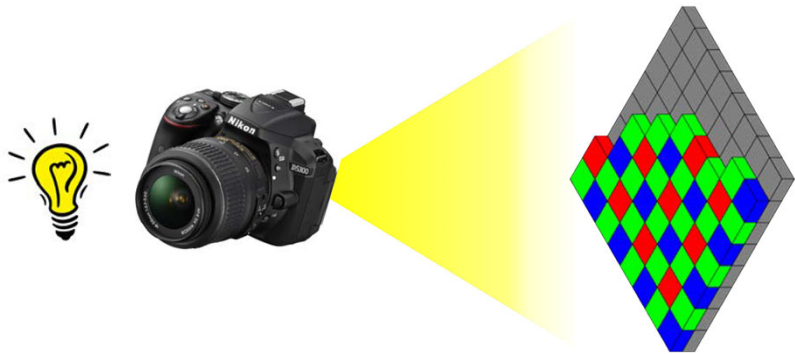
Raster Input Devices



$$i = 0.2126r + 0.7152g + 0.0722b.$$

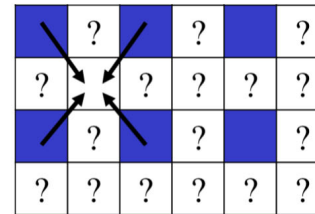
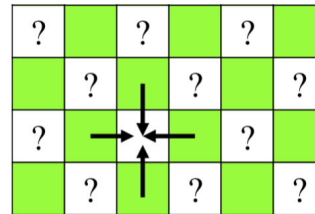
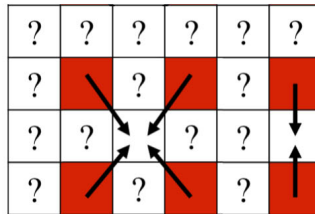
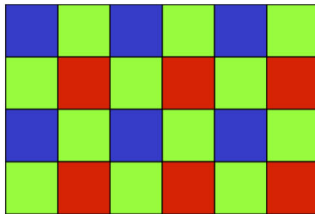


Why more 'green' than 'red' or 'blue'?

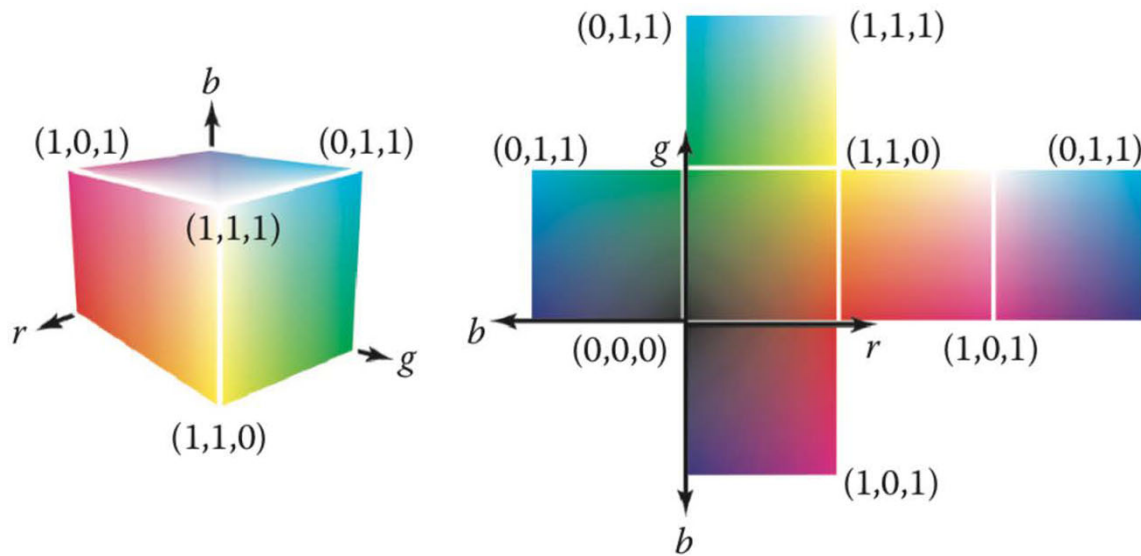


$$i = 0.2126r + 0.7152g + 0.0722b.$$

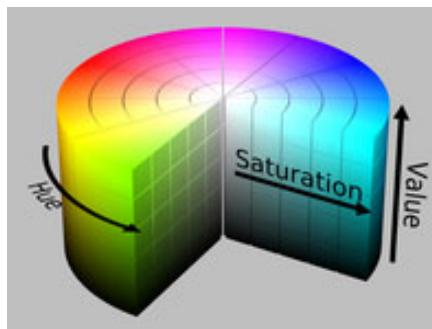
Demosaicing an image



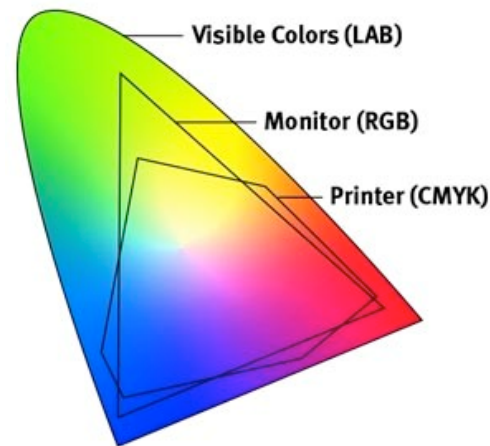
RGB Images



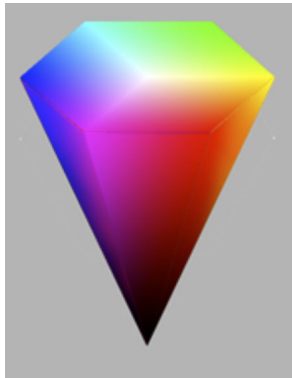
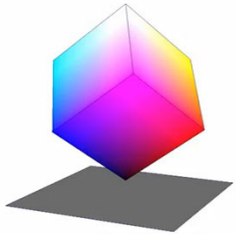
black = $(0, 0, 0)$,
 red = $(1, 0, 0)$,
 green = $(0, 1, 0)$,
 blue = $(0, 0, 1)$,
 yellow = $(1, 1, 0)$,
 magenta = $(1, 0, 1)$,



HSV



HSV <=> RGB



chroma

$$M = \max(R, G, B)$$

$$m = \min(R, G, B)$$

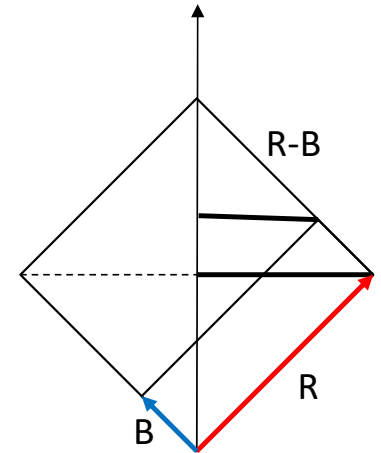
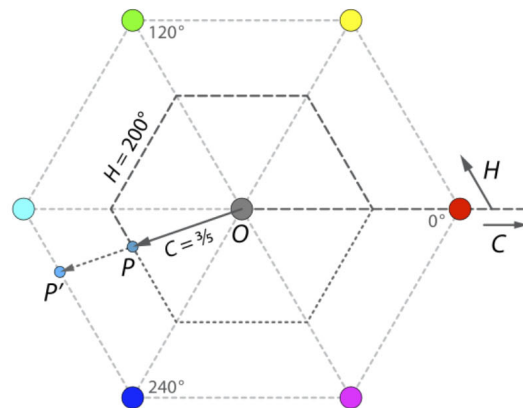
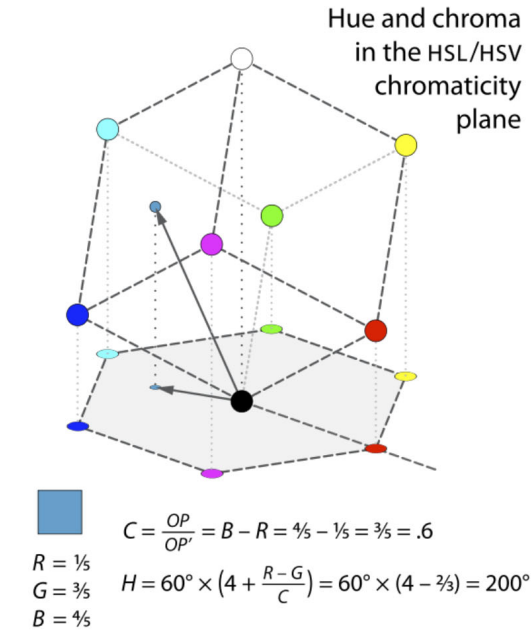
$$C = \text{range}(R, G, B) = M - m$$

$$V = \max(R, G, B) = M$$

$$S_V = \begin{cases} 0, & \text{if } V = 0 \\ \frac{C}{V}, & \text{otherwise} \end{cases}$$

$$H' = \begin{cases} \text{undefined}, & \text{if } C = 0 \\ \frac{G-B}{C} \bmod 6, & \text{if } M = R \\ \frac{B-R}{C} + 2, & \text{if } M = G \\ \frac{R-G}{C} + 4, & \text{if } M = B \end{cases}$$

$$H = 60^\circ \times H'$$



$$V=R$$

$$S=(R-B)/R$$

RGB to HSV

$M = \max(R, G, B)$

$m = \min(R, G, B)$

In the HSV hex-cone:

V is the height of the color along the cone axis.

NOTE: The cone axis has all shades of grey i.e. $R=G=B$,

black $V=0$; white $V=1$;

all primary colors red, green, blue, cyan, magenta, yellow $V=1$,

And in general $V=M$

Let R be M and B be m (without loss of generality).

Then the RGB color is somewhere in the triangular wedge shown, and specifically somewhere in the equilateral yellow triangle (i, j, k) .

The yellow triangle vertex colors are $i=(R,0,0)$, $j=(R,R,0)$ and $k=(R,R,R)$.

Now the color $c=(R,G,B)$ must lie on the yellow dashed line (a,b) parallel to (i,j) , with the length of $(a,k)=R-B$.

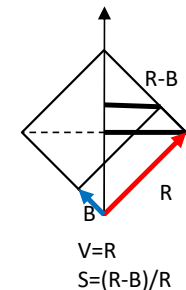
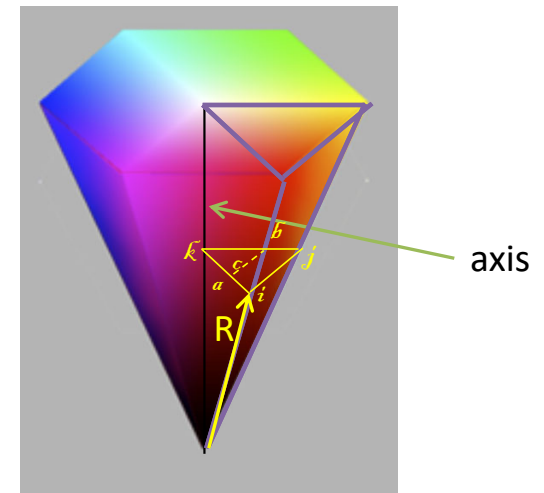
Saturation S ranges from 0 along the axis to 1 on the boundary of the cone, i.e. $S = |a,k| / |i,k| = (R-B)/R$.

Looking at the R,B axis ortho view at right can also provide intuition to why increasing B reduces saturation.

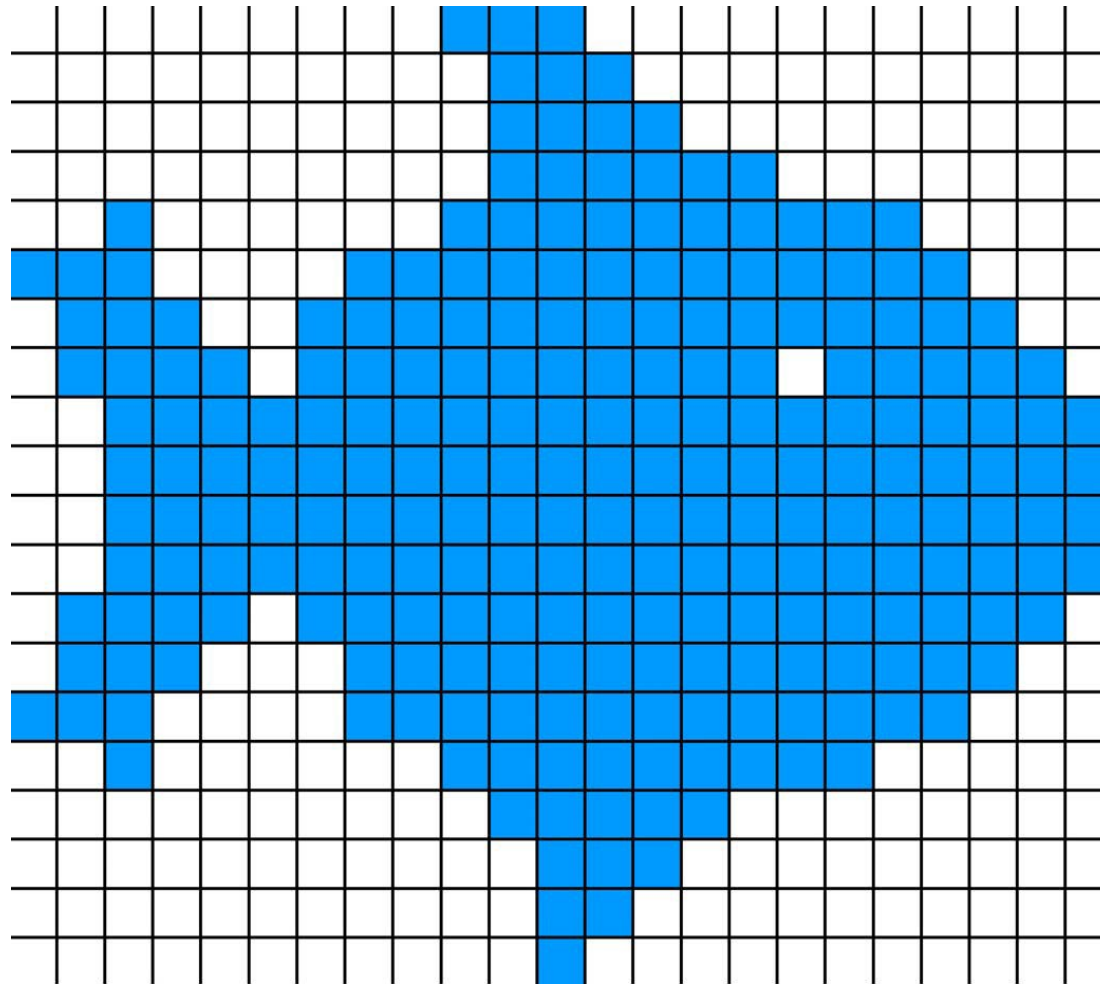
Hue is the angle around the hexagon. H is undefined if $S=0$. otherwise $= |c,a| / |a,b| = (G-B)/(R-B) * 60^\circ$.

Remember G is a value between B and R ($a=(R,B,B)$ and $b=(R,R,B)$).

The mod / +2 /+4 places the triangle in the right pie slice (out of 6) so the angle is $0..360^\circ$ instead of $0..60^\circ$.



Raster Image



Data Types for Raster Images

Storage for 1024x1024 image (1 megapixel) = $2^{10} \times 2^{10}$ pixels

bitmap 1 bit per pixel (bpp) : 2⁷KB or 128KB (2³ pixels per byte)

grayscale 8bpp: 1MB

grayscale 16bpp: 2MB

color RGB 24bpp: 3MB

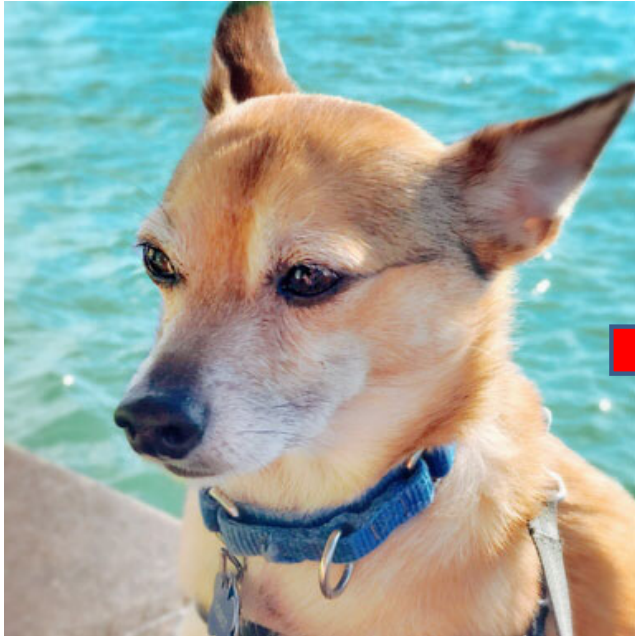
color + alpha (transparency) 32bpp: 4MB

floating-point HDR color: 12MB (each of 3 color channels is a 32bit float i.e. 4B)

Q: Suppose you have a 767 X 772 rgb image stored in an array called `data`. How would you access the green value at the pixel on the 36th row and 89th column?

A: `data[1 + 3*(88+767*35)]` (Remember C++ starts counting with 0).

Assignment #1 tasks



`src/reflect.cpp`

Horizontally reflect an image (like a mirror)

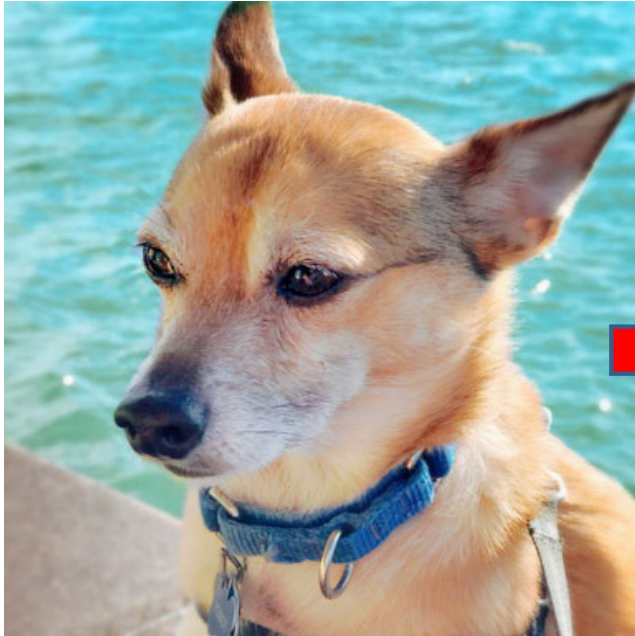


`src/rotate.cpp`

Rotate an image 90° counter-clockwise



Assignment #1 tasks



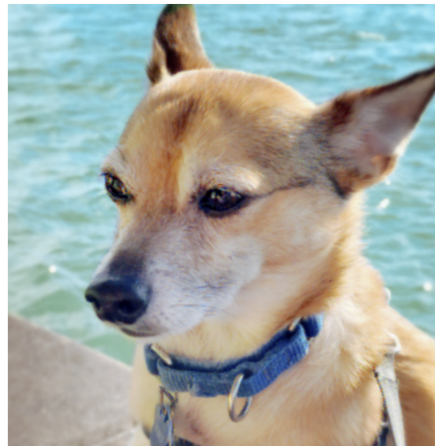
`src/rgb_to_gray.cpp`

Convert a 3-channel RGB image to a 1-channel grayscale image.



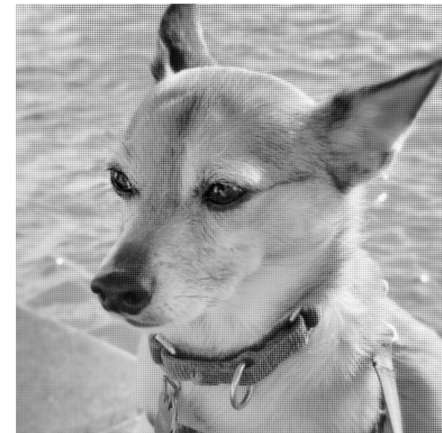
`src/demosaic.cpp`

Given a mosaiced image (interleaved GBRG colors in a single channel), created a 3-channel rgb image.

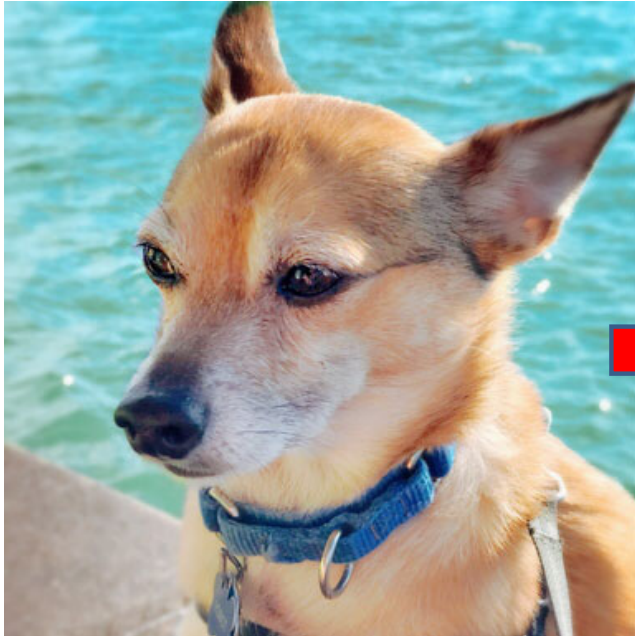


`src/simulate_bayer_mosaic.cpp`

Simulate an image acquired from the Bayer mosaic by taking a 3-channel rgb image and creating a single channel grayscale image composed of interleaved red/green/blue channels. The output image should be the same size as the input but only one channel.



Assignment #1 tasks



`src/rgb_to_hsv.cpp`

Convert a color represented by red, green and blue intensities to its representation using hue, saturation and value.

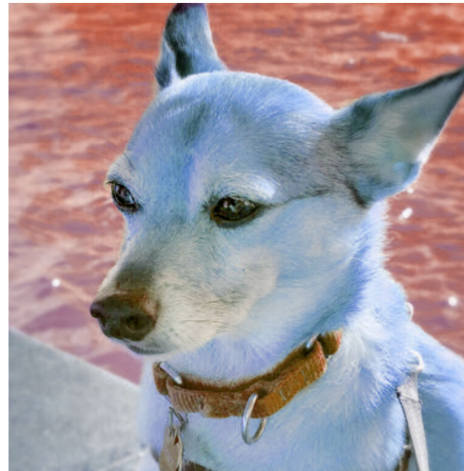
`src/hsv_to_rgb.cpp`

Convert a color represented by hue, saturation and value to its representation using red, green and blue intensities.

`src/hue_shift.cpp`

Shift the hue of a color rgb image.

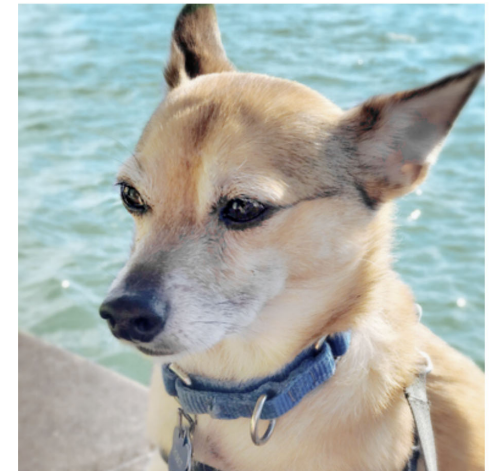
Hint: Use your `rgb_to_hsv` and `hsv_to_rgb` functions.



`src/desaturate.cpp`

Desaturate a given rgb color image by a given factor.

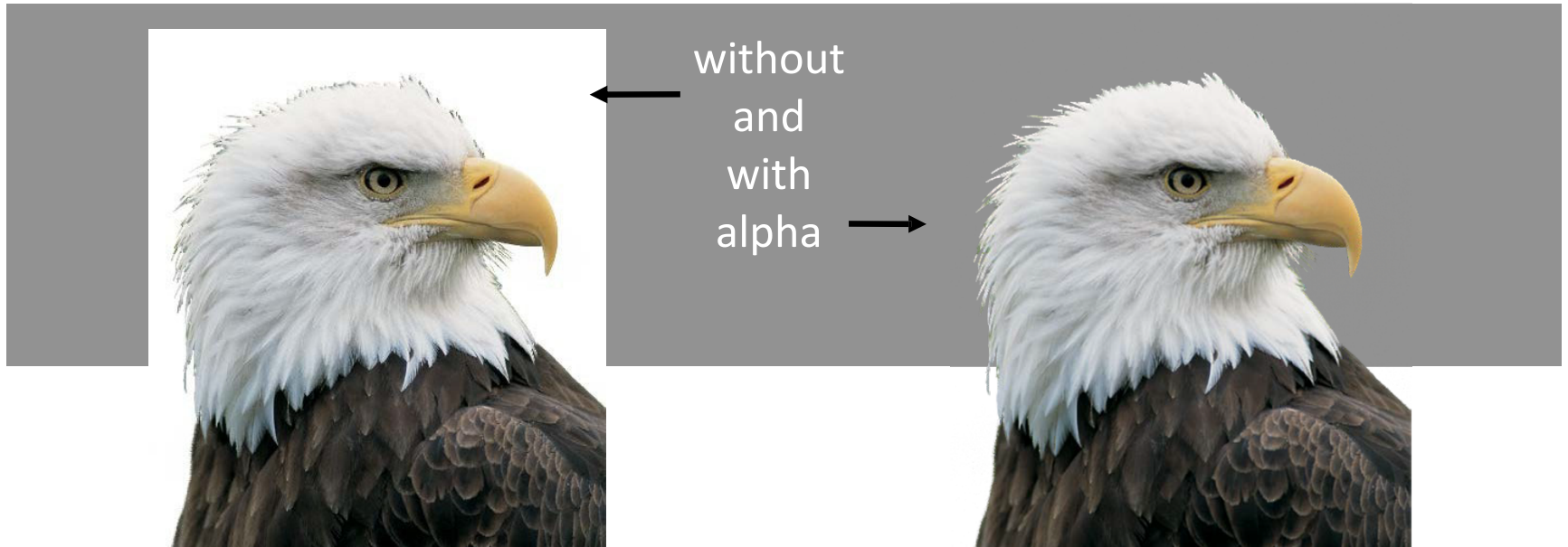
Hint: Use your `rgb_to_hsv` and `hsv_to_rgb` functions.



Transparency

Append (Red, Green, Blue)

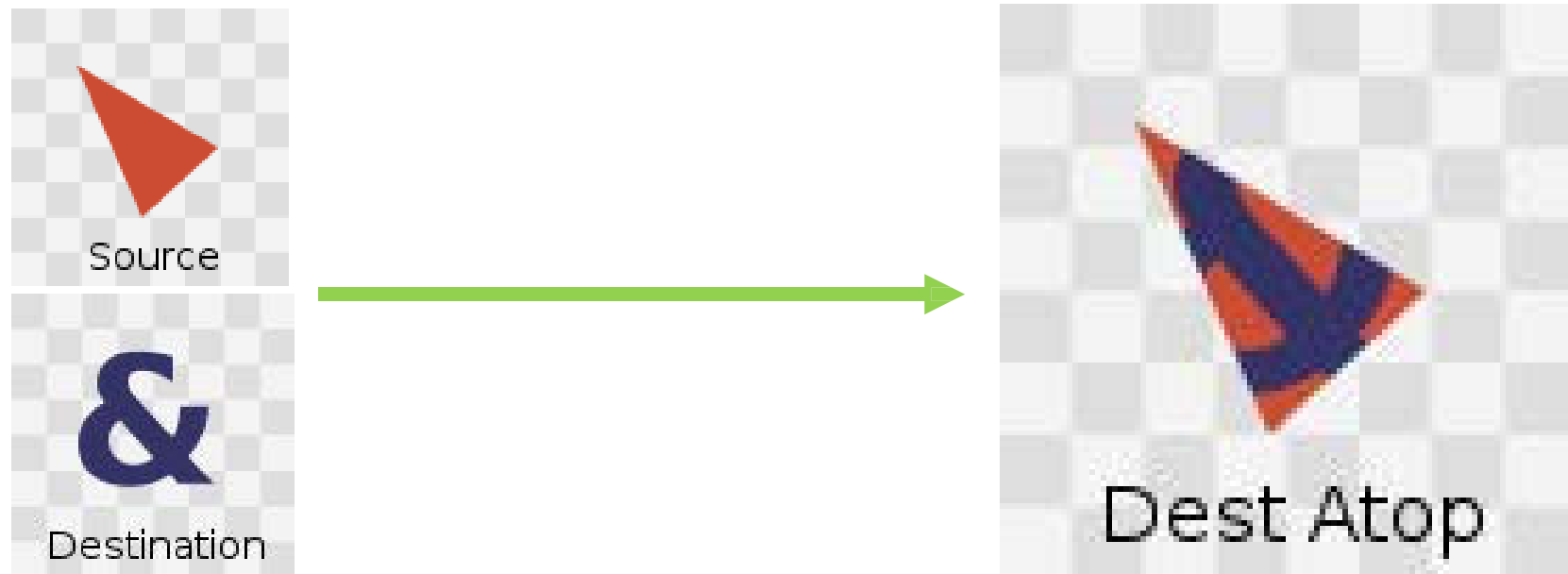
to be (Red, Green, Blue, Alpha)



$$\mathbf{c} = \alpha \mathbf{c}_f + (1 - \alpha) \mathbf{c}_b.$$

Compositing

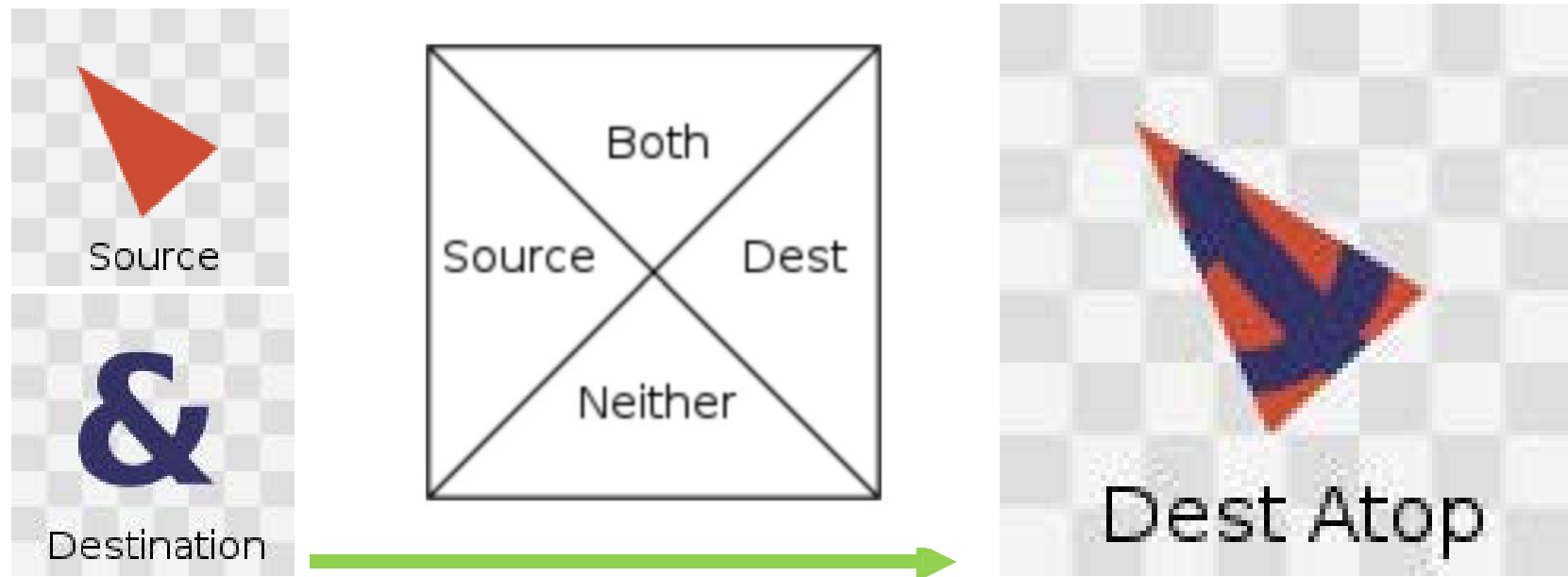
Compositing is about layering images on top of one another



<http://ssp.impulsetrain.com/porterduff.html>

Compositing

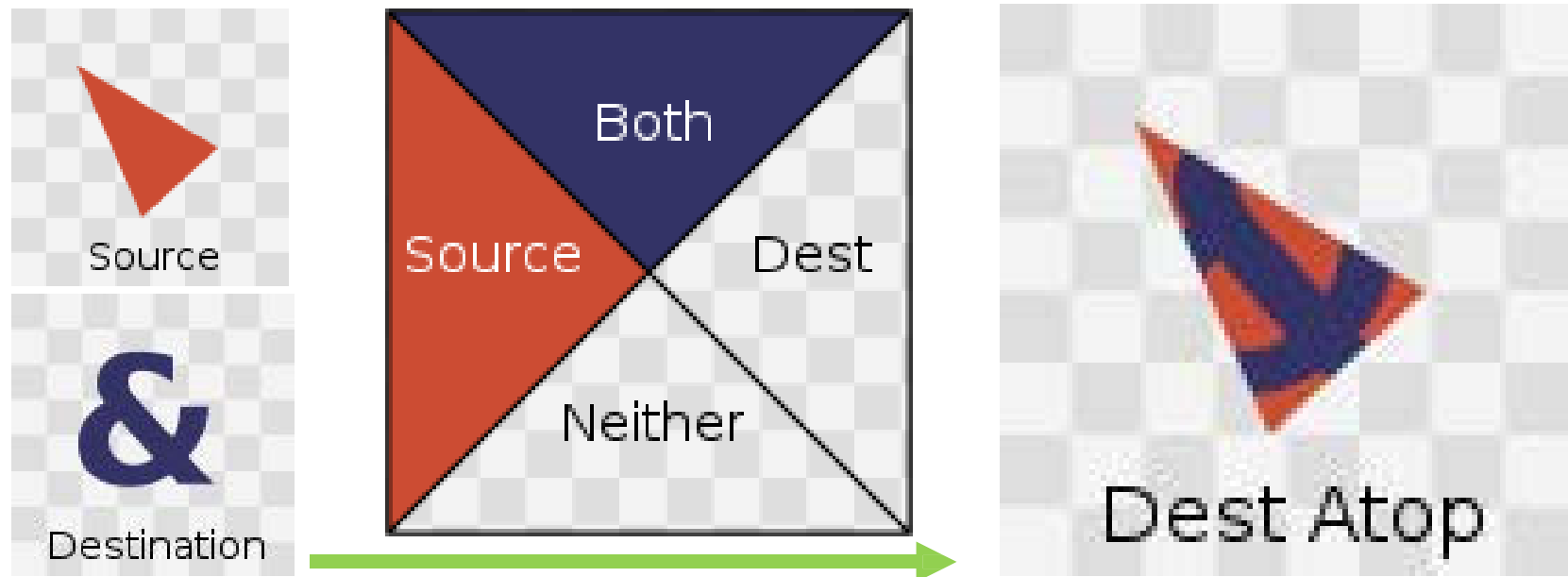
Compositing is about layering images on top of one another



<http://ssp.impulsetrain.com/porterduff.html>

Compositing

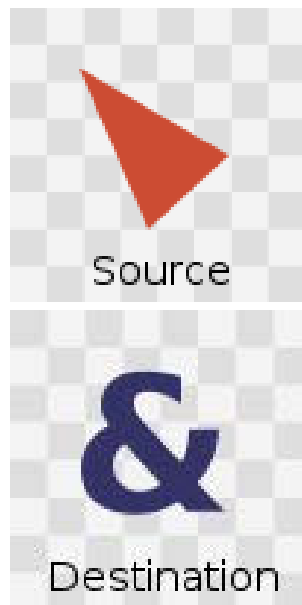
Compositing is about layering images on top of one another



<http://ssp.impulsetrain.com/porterduff.html>

Compositing

Compositing is about layering images on top of one another



$$A_{\text{src}} \cdot [s] + A_{\text{dst}} \cdot [d] + A_{\text{both}} \cdot [b]$$

$$A_{\text{src}} = \alpha_s \cdot (1 - \alpha_d)$$

$$A_{\text{dst}} = \alpha_d \cdot (1 - \alpha_s)$$

$$A_{\text{both}} = \alpha_s \cdot \alpha_d$$

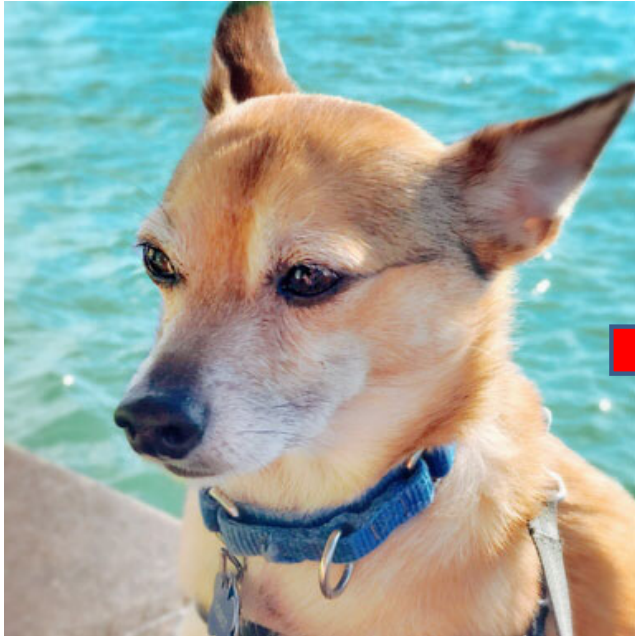
	[s]	[d]	[b]
Src	<i>s</i>	0	<i>s</i>
Atop	0	<i>d</i>	<i>s</i>
Over	<i>s</i>	<i>d</i>	<i>s</i>
In	0	0	<i>s</i>
Out	<i>s</i>	0	0
Dest	0	<i>d</i>	<i>d</i>
DestAtop	<i>s</i>	0	<i>d</i>
DestOver	<i>s</i>	<i>d</i>	<i>d</i>
DestIn	0	0	<i>d</i>
DestOut	0	<i>d</i>	0
Clear	0	0	0
Xor	<i>s</i>	<i>d</i>	0

→ $B(s, d)$ for color blending



<http://ssp.impulsetrain.com/porterduff.html>

Assignment #1 tasks



`src/over.cpp`

Compute $C = A \text{ Over } B$, where A and B are semi-transparent rgba images and "Over" is the Porter-Duff Over operator.



Raster Image issues: Gamma Correction



Linear encoding $V_S =$	0.0	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1.0
Linear intensity $I =$	0.0	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1.0

Display intensity is nonlinear wrt input intensity

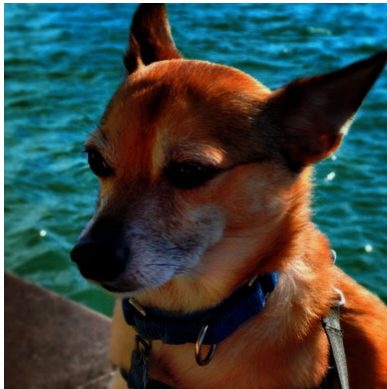
Raster Image issues: Gamma Correction

$$\text{displayed intensity} = (\text{maximum intensity}) a^\gamma$$

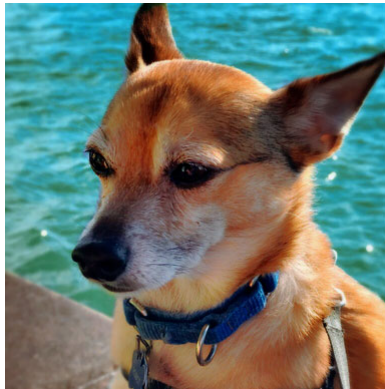
... of display image amplitude [0,1] gamma

Measuring Gamma for a display:

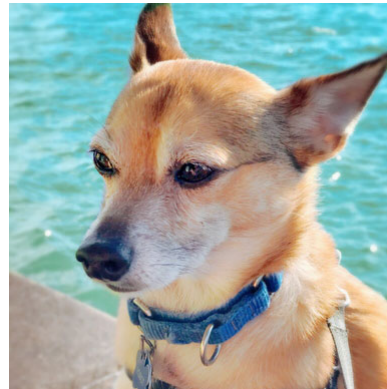
find image amplitude a for $\frac{1}{2}$ display brightness $\Rightarrow \gamma = \frac{\ln 0.5}{\ln a}$



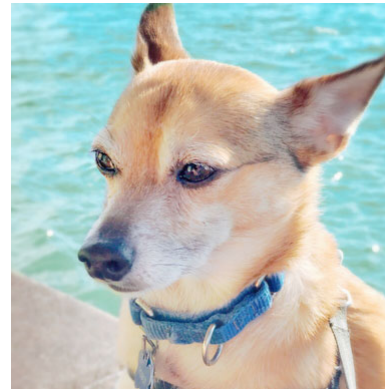
$\gamma=0.25$



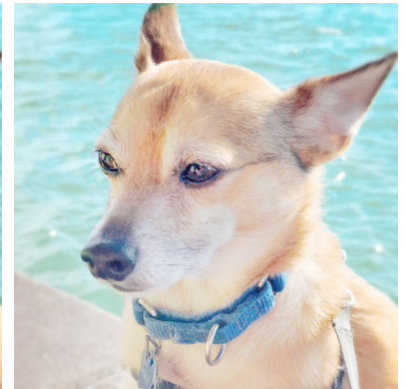
$\gamma=0.5$



$\gamma=1.0$

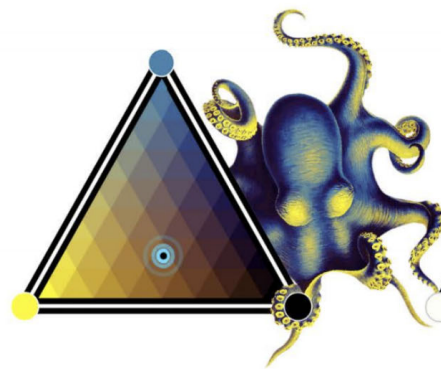
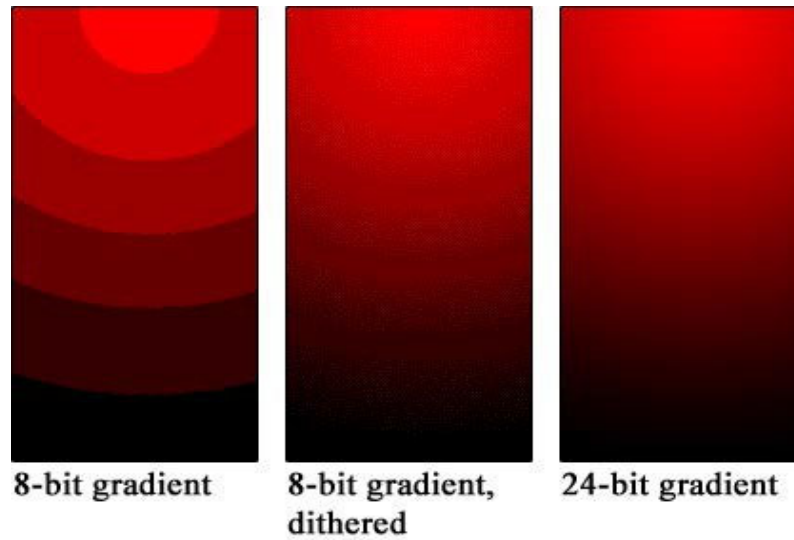


$\gamma=1.5$

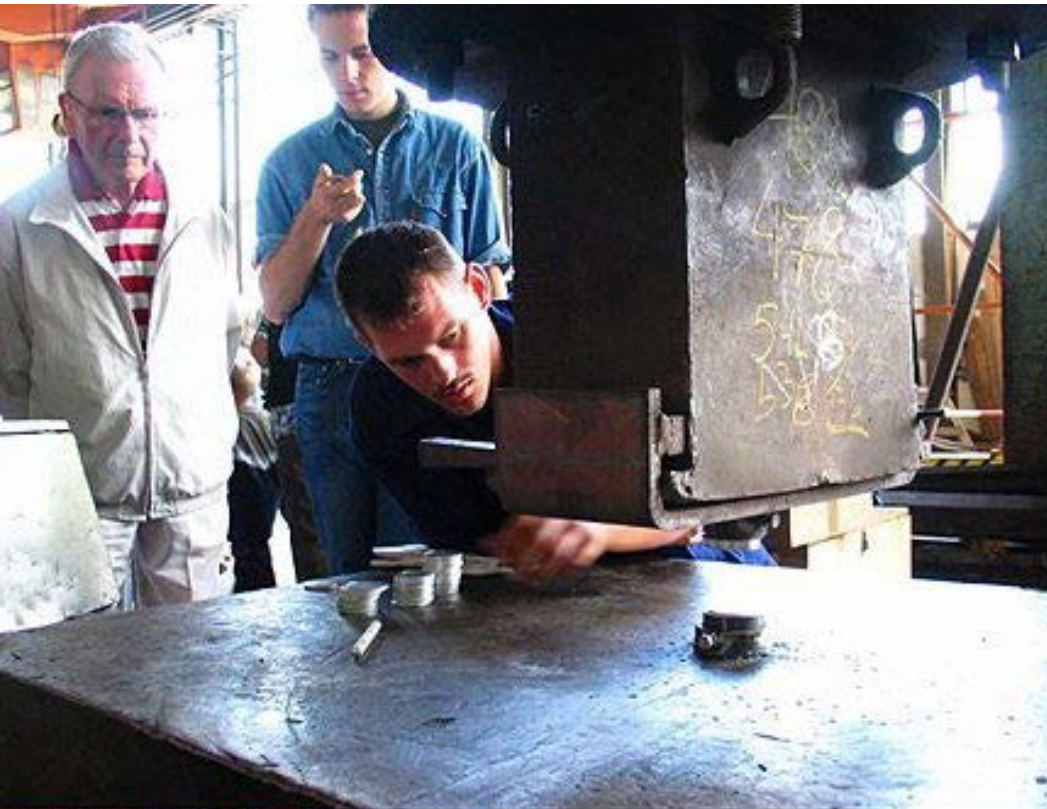


$\gamma=2.0$

Raster Image issues: Banding



Raster Image issues: Clipping



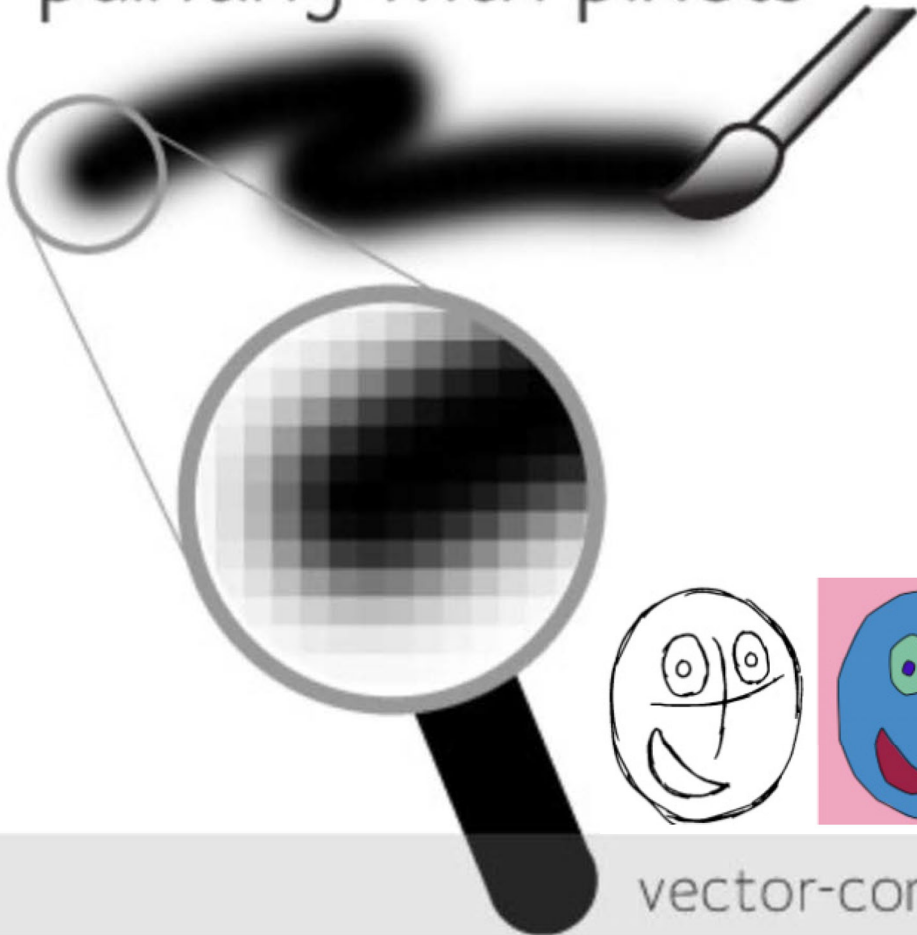
Original



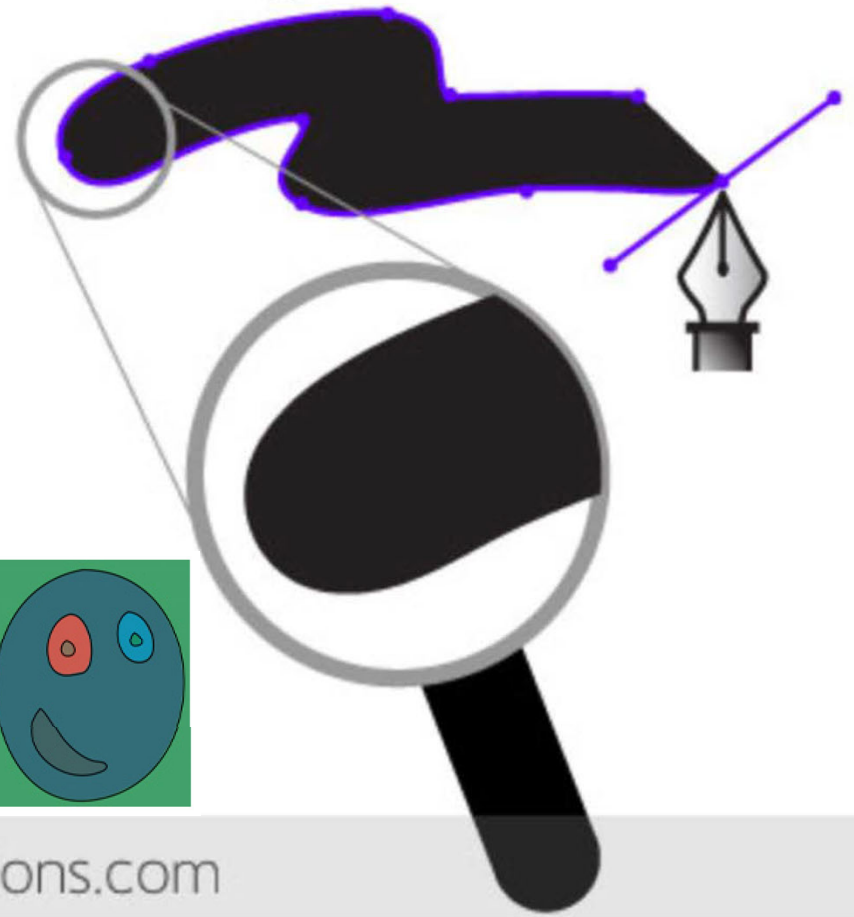
Clipped

Aside: More Than Just Raster Images

painting with pixels



drawing with vectors



vector-conversions.com

Assignment 1

Available Right Now

Next Week: Ray Casting